

Transport and Telecommunication Institute (TSI)

SkyTracker: Real-Time Flight Radar & Telemetry System

Final Project Presentation — Web Application Construction Course

Author: Artyom Vorontsov | **Tech Stack:** React, NestJS, PostgreSQL, SASS, OpenSky API

1. Project Goal & Overview

- **Objective:** Construct a high-performance dynamic web application to track live commercial aircraft globally.
- **Live Telemetry Data:** Integrates directly with OpenSky Network REST API for real-time state vectors.
- **Modern UI/UX:** Implements a responsive dark glassmorphic radar dashboard built in React & SASS.
- **Persistence & API:** Powered by a NestJS REST API with PostgreSQL database caching and user watchlists.

2. System Architecture & Tech Stack

Layer	Technologies & Libraries	Key Role
Frontend	React 18, TypeScript, Leaflet, SASS	Interactive map UI, real-time telemetry rendering
Backend API	NestJS, RxJS, Axios, Swagger OpenAPI	OpenSky ingestion, REST endpoints, telemetry stats
Database	PostgreSQL, TypeORM	Persistent flight logs & user watchlists
DevOps	Docker, Docker Compose, Nginx	Containerized local execution & hosting configs

3. Demonstration of 5 Dynamic Elements

1. **Dynamic Radar Map:** Real-time moving plane markers rotated live based on heading degrees.
2. **Aircraft Telemetry Inspector:** Live side panel inspecting speed, altitude, and vertical rate of selected planes.
3. **Airspace Statistics Aggregator:** Metrics card dynamically calculating average speed and airborne totals.
4. **Multi-Attribute Airspace Filter:** Instant callsign/country search and airborne-only filter toggle.
5. **Pinned Watchlist Manager:** Interactive bookmarking persisting user-tracked aircraft to PostgreSQL.

4. Advanced Styling & Web Best Practices

- **SASS Preprocessor:** Utilized custom variables, nested selectors, and ``@mixin glass-panel`` for clean styling.
- **RESTful API Standards:** NestJS controller endpoints following OpenAPI specifications with Swagger docs.
- **Resilience & Fallback:** Graceful degradation with dynamic flight drift simulation if API rate limits occur.
- **Production Readiness:** Docker Compose orchestrates database, backend, and static Nginx frontend seamlessly.

Project Summary & Final Results

- ✓ Complete dynamic web application delivered according to TSI guidelines.
- ✓ Source code packaged, documented, and ready for lecture presentation.

Thank you for your attention! Questions?